Title: A 32-bit RISC-V ASIC implementation using Chisel

A DISSERTATION SUBMITTED IN PARTIAL FULFILMENT
FOR THE DEGREE OF

Master of Technology

IN THE
FACULTY OF ENGINEERING

BY

SANIDHYA SAXENA, TOMS JIJI VARGHESE

GUIDED BY

KURUVILLA VARGHESE

DEPARTMENT OF ELECTRONIC SYSTEMS ENGINEERING

INDIAN INSTITUTE OF SCIENCE, BANGALORE

MAY 2024

Contents

Chapter 1

Pre-study

This report details the pre-study conducted to understand the design and architecture of a RISC-V processor and its implementation using the Chisel hardware construction language. We embarked on a comprehensive exploration through:

- **Market Survey:** We investigated existing RISC-V-based controllers to identify industry trends, potential application areas, and current performance benchmarks.
- **Product Survey:** We analyzed commercially available controllers (if applicable) to understand their functionalities, resource utilization, and potential limitations.
- **Wish List Specification:** Based on the market and product surveys, we formulated a detailed wish list outlining the desired features and capabilities of our RISC-V controller.

1.1 Chisel: A Powerful Tool for Hardware Design

The pre-study identified Chisel as a compelling open-source hardware description language (HDL) for constructing digital circuits and electronics at the register-transfer level (RTL). Chisel stands for "Constructing Hardware in a Scala Embedded Language," reflecting its innovative approach of leveraging Scala syntax for hardware design. This offers several advantages:

- **Expressive Syntax**: Chisel inherits the expressiveness and flexibility of Scala, allowing for concise and readable hardware descriptions. Compared to traditional HDLs like Verilog or VHDL, Chisel can simplify complex hardware constructs, improving design clarity and maintainability.

- **Functional Programming Paradigm**: Chisel embraces the functional programming paradigm, promoting immutability and promoting hardware components as functions of their inputs. This fosters a more modular design approach, making it easier to decompose and reason about complex digital systems.
- **Integration with Scala Ecosystem**: Chisel seamlessly integrates with the rich Scala ecosystem, allowing developers to leverage existing libraries and tools for tasks like testing, verification, and simulation. This can streamline the hardware design process and enhance overall development efficiency.

To effectively utilize Chisel's capabilities, a development environment was established on a Linux operating system. This involved installing and configuring essential tools like:

- **Scala Compiler**: The Scala compiler is required to translate Chisel code into its underlying hardware representation.
- **Chisel Libraries**: Specific Chisel libraries provide essential building blocks and functionalities for hardware design.
- **Simulation Tools**: Simulation tools like FIRRTL allow for testing and verification of Chisel designs before hardware synthesis.

Following the establishment of the development environment, the pre-study progressed to focus on the design of fundamental digital building blocks. These elementary components constitute the cornerstone upon which more intricate digital systems are constructed.

- **Counters**: These circuits execute incrementing or decrementing operations, serving a critical role in functionalities such as timing and sequencing within a digital system.
- **Multiplexers**: Multiplexers possess the capability of selecting a single data input from a collection of potential inputs based on a designated control signal. This functionality allows for flexible data routing within the system.
- **Finite State Machines (FSMs)**: FSMs are sequential circuits that transition between distinct states in response to both input and output signals. They play a central role in numerous digital systems, forming the core logic for controlling their behavior.

The process of designing and implementing these fundamental elements not only facilitated a comprehensive comprehension of Chisel's functionalities but also established a firm grasp of the core principles that govern digital hardware design.

1.2 Market Survey

RISC-V (RISC Five), an open-source instruction set architecture (ISA) is rapidly transforming the computing landscape. Unlike traditional proprietary ISAs, RISC-V offers a royalty-free and modular standard. This fosters a wave of innovation across diverse computing domains, including the Internet of Things (IoT), edge computing, and Artificial Intelligence (AI). RISC-V's low-power consumption and inherent scalability make it ideal for resource-constrained IoT devices. Additionally, its flexible nature allows for customization to specific processing needs encountered at the edge of computing networks, and its modularity facilitates the development of specialized hardware accelerators for AI applications.

1.2.1 Market Trends

The pre-study revealed a burgeoning market for RISC-V processors, driven by several key trends:

- **Soaring Adoption**: Across diverse sectors, the open nature of RISC-V is fueling rapid adoption. This openness allows for customization, reduces time to market, and provides cost savings for developers.
- **Application Expansion**: Initially popular in embedded systems, RISC-V is now making inroads into high-performance computing domains such as servers, networking equipment, and even some mobile devices. This diversification showcases its growing versatility.
- **Thriving Ecosystem**: The RISC-V ecosystem is flourishing with the development of essential tools like software tools, compilers, operating systems, and hardware platforms. This robust ecosystem fosters innovation and further adoption.
- **Industry Backing**: Major tech giants, including Google, NVIDIA, Western Digital, and Alibaba, are lending their support to RISC-V. This industry backing signifies its potential to become a mainstream architecture in the future.

1.2.2 Growing Demand for Customization:

RISC-V's open architecture is a key driver behind its growing adoption. This openness unlocks a new level of customization for companies developing Application-Specific Integrated Circuits

(ASICs). Beyond the traditional benefits of ASICs, RISC-V empowers a more granular level of control, allowing for:

- **Tailored Solutions**: Traditional ASICs offer customization, but RISC-V's open architecture unlocks a new level. Companies can fine-tune their ASIC designs, optimizing performance, power efficiency, and other critical metrics to their exact application requirements.
- **Industry-Specific Optimization**: Different industries have unique needs. RISC-V empowers companies to develop bespoke solutions that meet the stringent demands of sectors like IoT, automotive, aerospace, and telecommunications. By incorporating RISC-V cores tailored to their specific needs, companies can achieve significant performance and efficiency gains.

1.2.3 Booming Ecosystem Supports RISC-V ASIC Design

RISC-V's open architecture fosters not only hardware customization but also the development of a comprehensive ecosystem. This ecosystem plays a critical role in facilitating the broad adoption of RISC-V within ASIC design by providing essential tools and resources that streamline the development process. An examination of the following elements reveals the key components of this maturing ecosystem:

- **Comprehensive Development Tools**: A growing suite of development tools caters specifically to RISC-V-based ASICs. This includes tools for Register Transfer Level (RTL) design, verification, synthesis, and physical design, ensuring a smooth workflow for ASIC designers.
- **Diverse IP Core Offerings**: Companies specializing in RISC-V IP cores are providing a rich set of options for ASIC integration. These offerings encompass various configurations of RISC-V cores, peripheral IP (Intellectual Property) blocks, and interconnect IP. This comprehensive library empowers ASIC designers to efficiently build complex systems-on-chip (SoCs) tailored to their specific needs.

1.2.4 Prominent Players in the Maturing RISC-V Ecosystem

The pre-study identified several prominent players shaping the maturing RISC-V ecosystem:

- **SiFive**: A leading provider of RISC-V IP cores, SiFive offers a comprehensive portfolio of customizable processor designs catering to diverse application needs.
- **Andes Technology**: This company specializes in developing high-performance, low-power RISC-V processor cores for a wide range of applications, including those in the Internet of Things (IoT), automotive, and Artificial Intelligence (AI) sectors.
- **NVIDIA**: Following its acquisition of Arm, NVIDIA's growing interest in RISC-V signifies its potential as a viable open-source architecture.
- **Microchip Technology**: Microchip contributes to the RISC-V ecosystem by offering microcontrollers based on the RISC-V architecture, targeting applications in the IoT and embedded systems domains.

1.2.5 Acceleration of Prototyping and Time-to-Market:

RISC-V's open architecture offers significant advantages in terms of development time and prototyping for ASIC design:

- **Streamlined Prototyping**: The open nature of RISC-V allows designers to experiment with various configurations and architectural choices readily. This flexibility fosters faster prototyping cycles, enabling rapid design exploration and optimization. As a result, companies can iterate on their designs quickly, accelerating the product development process.
- **Agile Development Compatibility**: RISC-V aligns well with agile development methodologies. Its open nature facilitates quick modifications and easier integration of feedback from stakeholders during the design phases. This iterative approach reduces the time needed to refine the design and ultimately shortens the time-to-market for ASIC products.

1.2.6 Cost-Effectiveness and Scalability with RISC-V

RISC-V presents compelling advantages for ASIC design in terms of both cost and scalability:

- **Reduced Development Costs**: Unlike proprietary architectures that require licensing fees, RISC-V's open-source nature eliminates these upfront costs, making it a more cost-effective option for developers.

- **Scalable Architecture**: RISC-V offers remarkable scalability. The same core architecture can be adapted to create a wide range of solutions, from resource-constrained, low-power designs ideal for IoT devices to high-performance processors suitable for data centers. This scalability allows ASIC designers to leverage a familiar architecture while tailoring it to meet the specific needs of their application without significant architectural overhauls.

1.2.7 Challenges and Considerations for RISC-V Adoption

While RISC-V presents enticing opportunities, it's crucial to acknowledge the challenges associated with its adoption in ASIC design:

- **Established Market Competitors**: The dominance of entrenched architectures like x86 and Arm in specific markets presents a significant hurdle for RISC-V's widespread adoption. Overcoming these established players and their mature ecosystems requires ongoing innovation and industry support for RISC-V.
- **Intellectual Property Considerations**: Although RISC-V boasts an open-source core, challenges related to intellectual property can still arise when integrating RISC-V cores into commercial products. Careful assessment and management of IP licensing terms associated with specific RISC-V implementations are necessary.
- **Verification and Validation Complexity**: Verifying complex ASIC designs remains a significant challenge, regardless of the chosen ISA. Integrating RISC-V processors necessitates robust verification strategies to ensure the design's correctness and reliability. Designers need to invest in thorough verification methodologies for RISC-V-based ASICs.
- **Evolving Ecosystem**: The RISC-V ecosystem, while rapidly maturing, lacks complete maturity in certain areas. Aspects like optimized libraries, established design flows, and comprehensive tool support are still under development. ASIC designers must carefully evaluate the maturity of the RISC-V ecosystem in relation to their specific needs and project timelines.

1.3 Wish Specification

This project focuses on developing a RISC-V core specifically designed for embedded systems and custom hardware projects. The core prioritizes achieving a balanced trade-off between three critical factors:

• **Performance**: The core should deliver efficient processing capabilities.

• **Power Consumption**: Low power consumption is crucial for battery-powered applications.

• **Area Footprint**: Minimizing the core's physical size on a chip optimizes cost and resource utilization.

The RISC-V core will leverage a Harvard architecture to enable simultaneous instruction and data memory accesses. This approach enhances overall processing efficiency. Additionally, an optimizing folded 6-stage pipeline will be implemented. This pipeline maximizes overlap between execution stages and memory accesses, reducing stalls and improving performance.

Optional features include Branch Prediction, Instruction Cache, Data Cache, Debug Unit and optional Multiplier/Divider Units. Parameterized and configurable features include the instruction and data interfaces, the branch-prediction-unit configuration, and the cache size, associativity, replacement algorithms and multiplier latency. Providing the user with trade offs between performance, power, and area to optimize the core for the application.

1.3.1 Key Specifications:

1.3.1.1 High Performance 32-bit CPU design with Optional Extensions

- **Parameterized 32/64bit data**: The core will be a high-performance 32-bit CPU design with the option for parameterized 32/64-bit data handling.
- **Fast and Precise Interrupts**: The core prioritizes efficient interrupt handling, ensuring timely responses to critical events within the system.
- **Single-Cycle Execution (Optional)**: Single-cycle execution can be included as an option for applications requiring the highest performance. However, this approach may require careful consideration of trade-offs with power consumption and area footprint.
- **Optimizing Folded 6-Stage Pipeline**: This core implements an optimized folded 6-stage pipeline to maximize overlap between instruction execution and memory access stages.

This approach minimizes stalls and improves overall processing efficiency.

- **Memory Protection Support**: The core will offer memory protection support for enhanced system security.
- **Optional/Parameterized Caches**: Instruction and data caches can be optionally integrated to reduce memory access latency for performance-critical applications. The size, associativity, and replacement algorithms of these caches will be configurable.

1.3.1.2 Area and Power Efficiency

The design will prioritizes both minimizing the core's physical footprint (area) and reducing its power consumption:

- **Parameterized Data Width**: The data path width can be configured, allowing users to optimize the balance between performance and power consumption for their specific application. Wider data paths can process more data per cycle but require more hardware resources and consume more power.
- **Clock Gating Logic**: The core will employ clock gating logic to reduce dynamic power consumption. This technique essentially shuts off the clock signal to inactive portions of the core, minimizing unnecessary power usage.
- **Small Silicon Footprint**: The design prioritizes a compact layout to minimize the core's physical size on the chip. This not only reduces manufacturing costs but also allows for more efficient integration with other components on the same die.

1.3.1.3 Multicycle Architecture approach

- **Reduced Area Footprint**: Due to the absence of complex pipeline hazard detection and resolution logic, multicycle architectures require less silicon area. This can be crucial for resource-constrained embedded systems or designs targeting low-cost fabrication processes.
- **Synchronous Memory Compatibility**: Multicycle architectures are well-suited for use with synchronous memory, which is the prevalent type of memory found in most real-world applications, including FPGAs with block RAM. Synchronous memory offers reliable data transfers by coordinating operations with a clock signal.

- **Pipelined Architecture**: Pipelined designs increase performance by having several instruction "in fly" at the same time. But this also means there is some kind of "out-of-order" behavior: if an instruction at the end of the pipeline causes an exception all the instructions in earlier stages have to be invalidated.

1.3.1.4 Instruction Set Architecture (ISA)

The RISC-V core will support the base integer instruction set architecture (ISA) defined by the RISC-V specification. This core ISA provides a foundation for efficient execution of various workloads.

- **Basic Integer Instructions**: The core will support fundamental instructions for arithmetic, logical, data transfer, and control flow operations. This includes essential instructions like branch, jump, and call instructions for program branching and subroutine management.
- **Optional Branch Prediction**: A branch prediction algorithm can be optionally integrated to improve the efficiency of branch instructions. Branch prediction attempts to forecast the outcome of a conditional branch instruction, potentially reducing pipeline stalls and improving overall performance.
- **Custom ISA Extension Support (Optional)**: The design may offer the flexibility to incorporate user-defined RISC-V instruction extensions. These extensions can be tailored to specific application requirements, potentially accelerating performance for specialized tasks. However, the decision to include custom extensions requires careful consideration of development complexity and potential compatibility limitations.

Chapter 2

Study

2.1 Processor Design

The processors are all important components of computer architecture. Computer architecture is a specification that describes how hardware and software technologies are connected to create a computer platform. It refers to a system that processes any fetched instruction. There are many types of processor designs available and PPA tradeoffs are measured for each one of them and is then used in various applications. Instruction set architecture (ISA) defines program visible components and specifications it includes all the functions, capabilities, data formats, processor register etc. that are used by a computer system.

2.2 Instruction Set Architecture

ISA provides command to the processors, to tell what it needs to do. All processors can have same instruction set but still can have different internal design. In our project we'll be using the RISC-V ISA. RISC-V instruction set architecture (ISA) is now set to become a standard free and open architecture for academic and industrial applications [6]. We have both 32-bit and 64-bit RISC-V designs, and in this project we'll be going forward with 32-bit design.

The great interest on RISC-V comes from the fact that it is a free and open standard which allows anyone to easily develop their ideas. Furthermore RISC-V is modular, so it is based on a strong and frozen set, and all the extensions are optional without the need for continuous update as happens in incremental ISAs like the 80x86.

Comparision between various available ISA was made like MIPS, RISC, Intel, ARM [7]. RISC-V being an open source ISA, software development and ecosystem is available and the core can be used as an IP for various application development.

Despite the specific format used in the core we can identify four main subsets of instructions:

1. *Integer Computational Instructions*: They are encoded in R-type format or I-type format respectively if we have a register-register operation or a register-immediate operation. We have addition (ADD, ADDI), subtraction (SUB), shifts (SLL, SLLI, SRL, SRLI, SRA, SRAI), bitwise logical operations (AND, ANDI, OR, ORI, XOR, XORI) and comparison operations (SLT, SLTI, SLTU, SLTIU).

2. *Control Transfer Instructions:* These instructions are responsible for jumps to other instructions in the range of ±4 KiB that break the normal flow of the program. Jump And Link (JAL) and Jump And Link Integer (JALR) are dedicated to unconditional jumps and are encoded respectively with UJtype and I-type formats. Branch if Equal (BEQ), Branch if Lower Than (BLT), Branch if Lower Than Unsigned (BLTU), Branch if Not Equal (BNE), Branch if Greater or Equal (BGE), Branch if Greater or Equal Unsigned (BGEU). Note, BGT, BGTU, BLE, and BLEU can be synthesized by reversing the operands to BLT, BLTU, BGE, and BGEU, respectively.

3. *Load and Store Instructions:* These instructions are the only few simple instructions responsible for memory access. As RISC-V is a load-store architecture the only memory related operations are loading a value from memory to a register and storing a value from a register to the memory. Register to Memory instructions are those of the STORE family (SW, SH, SB) that store respectively a Word (32b), Half a word (16b) and a Byte (8b). The instructions responsible for Memory to Register transfers are those of LOAD family (LW, LH, LHU, LB, LBU), where the U stands for Unsigned as the Half (16b) and Quarter (8b) words are zero-extended instead of sign-extended as in the non-U version.

4. *Control and Status Register Instructions:* These are all SYSTEM instructions, encoded using I-type format. Control and Status Register atomic Read/Write (CSRRW) is used to atomically swap values in the CSRs and integer registers; atomic Read and Set/Clear bits (CSRRS, CSRRC) are used to read the value of a CSR and then set or clear specific bits of the read word. The corresponding immediate versions of the three previous instructions, CSRRWI, CSRRCI, and CSRRSI, behave like their counterparts but they use an immediate instead of an integer register.

2.2.1 Standard extensions

In the previous paragraph we have briefly described the base integer ISA of RISC-V but as known the most important feature of RISC-V is the possibility to easily add other custom instructions to enlarge the ISA and adapt it to specific objectives. Despite these "non-standard" extensions, there are several other standard ISA extensions that have been developed during years and can be added very easily to our particular implementation making the final RISC-V a very powerful core, and they are:

- "M" Standard Extension for Integer Multiplication and Division;
- "A" Standard Extension for Atomic Instructions;
- "F" Standard Extension for Single-Precision Floating-Point;
- "D" Standard Extension for Double-Precision Floating-Point;
- "Q" Standard Extension for Quad-Precision Floating-Point;
- "L" Standard Extension for Decimal Floating-Point;
- "C" Standard Extension for Compressed Instructions;
- "B" Standard Extension for Bit Manipulation;
- "J" Standard Extension for Dynamically Translated Languages;
- "T" Standard Extension for Transactional Memory;
- "P" Standard Extension for Packed-SIMD Instructions;
- "V" Standard Extension for Vector Operations;
- "N" Standard Extension for User-Level Interrupts;

2.3 Microarchitecture

Majority of the processor designs have 2 components

1. Data path
2. Control Path

Single cycle processor is a processor in which the instructions are fetched from the memory, and then executed, and the results are further stored in a single cycle, whereas in pipelined implementation, each step in execution takes one clock cycle. Since all instructions in a single cycle non pipelined structure do not necessarily take same amount of time rather the next instruction to be executed after an instruction can not start until the next clock cycle. Hence more resources will be reuired and concourrent execution is not possible.

Pipelined processor saves time by overlapping two instructionss' substages. Since, all RISC-V instructions are the same length, this makes it much easier to fetch instructions in the first pipeline stage and to decode them in the second stage. The more pipeline stages a processor has, the more instructions it can process at once and the less of a delay there is between completed instructions.

	Single Cycle	Pipelined Processor
Hardware Required	Less	More (Due to Registers)
CPI	1	>=1 (Due to Stalls and Hazards)
T_{clk}	Baseline	Less (Due to pipelining)
Throughput	1	>1
Power		

Table 2.1: Comparision between Single cycle vs Pipelined

We'll go with 5 stage pipelined architecture to achieve the benefit of maximum throughput and performance.

1. Instruction fetch
2. Intsrutcion Decode
3. Instruction Execute
4. Memory
5. Write Back

2.4 Other available designs

Many processor architecture, adopting RISC-V has been developed and many are under progress. RISC-V is an open - sourced instruction set infrastructure that allows a new realm of processor

creation by means of open collaborative domain, culminating in a pool of innovation with a 32- or 64-bit address space and a compressed ISA built for special application extensions[8].

Rocket[9] is a six staged open-source pipeline architecture with base as the RV64G ISA. The Rocket SOC generator is capable of generating multiple SOC design configuration prototypes from a single base instance. The Rocket scalar core, is a 64 bit 5-stage single-issue in-order pipeline that executes the RISC-V instruction set architecture (ISA).

Shakti-F series of processors are developed by a group of researchers from IIT Madras as an initiative to bring up computing units for space and nuclear research and development. Shakti-C, Shakti-T, Shakti-I, Shakti-E are variants of shakti computing core which are developed.

The RISC foundation serves as the foundation for ARM architecture, which similarly emphasizes simplicity and power economy. The load-store architecture, a combination of fixed-length 32-bit and variable-length Thumb instructions, and a significant amount of general-purpose registers are among ARM's core architectural features. These processors support cutting-edge functions including hardware virtualization, superscalar pipelines, and out-of-order execution[10].

2.5 Proposed Architecture

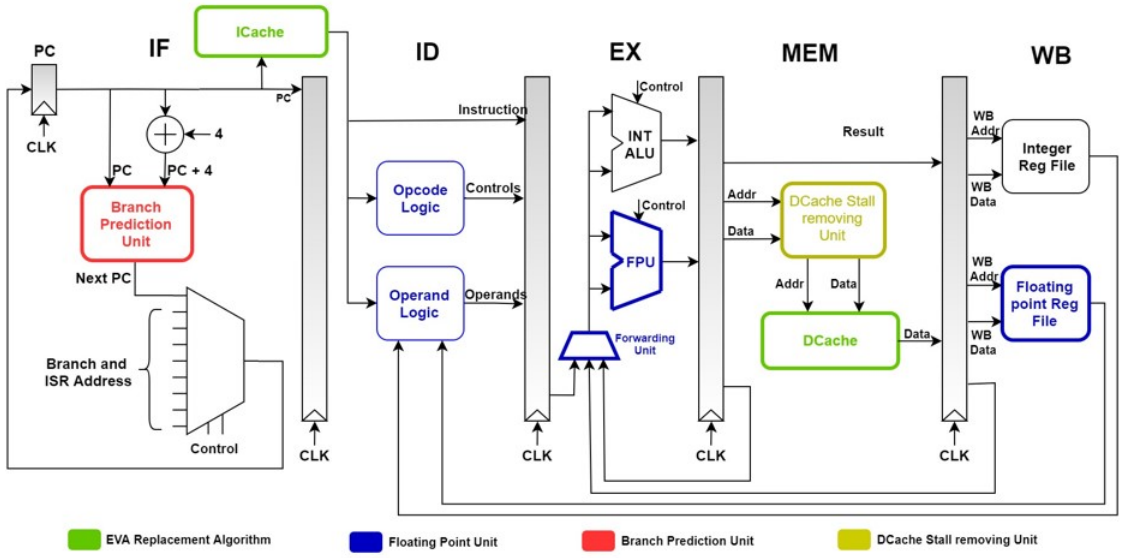

Figure 2.1: Proposed Architecture

1. **Instruction fetch:** The instruction is read from memory using the address in the PC and then is placed in the **IF/ID** pipeline register. This stage occurs before the instruction is identified.

2. **Instruction Decode**: The instruction in the **IF/ID** pipeline register supplies the register numbers for reading two registers and extends the sign of the immediate operand. These first two stages are executed by all instructions, since it is too early to know the type of the instruction.

3. **Instruction Execute**: The instruction reads the contents of a register and the sign-extended immediate from the **ID/EX** pipeline register and adds them using the ALU. That sum is placed in the **EX/MEM** pipeline register.

4. **Memory Access**: The instruction reading the data memory using the address from the EX/MEM pipeline register and loading the data into the MEM/WB pipeline register. If an instruction doesnt need to write the data in data memory (like R-Type instructions), data can be directly forwarded to MEM/WB pipeline register.

5. **Write-Back**: reading the data from the MEM/WB pipeline register and writing it into the register file in the middle of the figure.

2.6 Hazard Detection and Correction block

Due to data and control dependencies, in pipeline designs we have 2 major hazards:

1. *Data Hazard*

2. *Control Hazard*

When an instruction in the pipeline is dependent on the outcomes of a preceding instruction that hasn't yet entered the write back stage, it leads to data dependence. Control dependency occurs when control instructions are executed. The choice to branch or not branch for these instructions is made during the execution stage, after two consecutive instructions have entered the pipeline's fetch and decode stages. If no branch is chosen, the execution continues uninterrupted. However, if the branch is taken, the prior two instructions must be drained out before the current branch instruction can be executed, resulting in a control hazard.

1. *Data Forwarding to eliminate data hazard:* Ahead of the source instruction reaching write back stage, data is made available to the ALU for dependent instructions, which is known as forwarding.This data forwarding technique identifies the dependant instruction in pipeline and forwards the data to the dependant instruction thus eliminating data hazards in the pipeline. *We'll be using both IE-IE and Mem-IE data forwarding.*

2. *Adaptive dymanic Branch prediction to eliminate control hazard:* It enhances the accuracy of prediction by taking into account the recent behaviour of previous branches as well. If the instruction is a branch instruction it uses Branch Target Address, which is fetched during the Instruction fetch stage. It also makes use of a Local Prediction Table, which contains two bits that decide whether or not a branch is taken. In comparison to a one bit branch predictor, the branch predictor state changes only after two consecutive mispredictions, resulting in enhanced accuracy.

A RV32I RISC-V processor in this literature [11]uses data forwarding and dynamic branch predictor for handing the hazards.

2.6.1 Data Bypass design

Figure 2.2 shows the Fully bypassed datapath[12], which is used to bypass the result either from the IE/MEM stage or from MEM/WB stage based on the next intruction after decode stage.

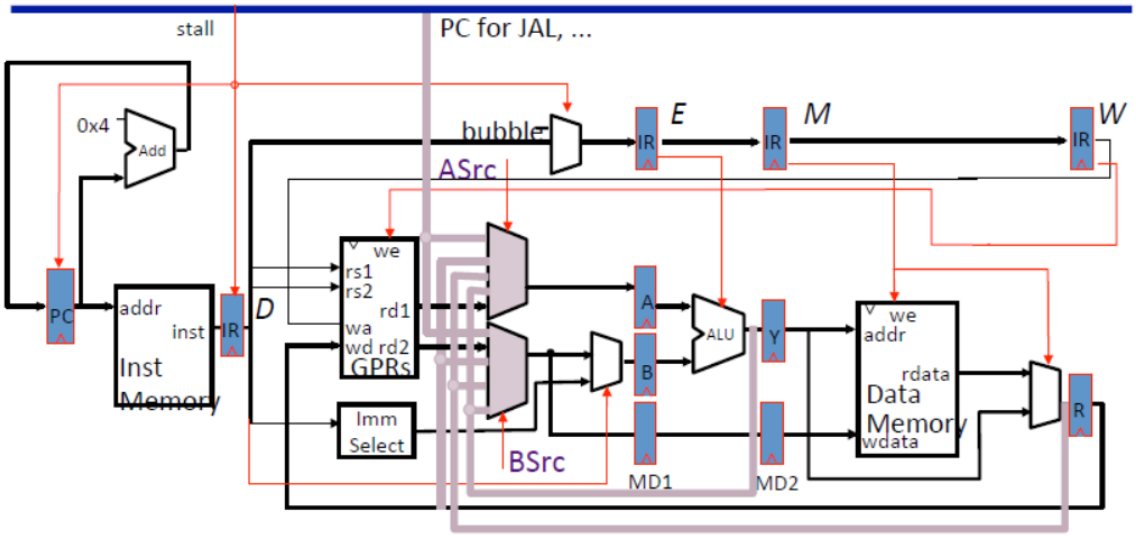

Figure 2.2: Bypass Datapath

If the current instruction is a ALU and ALUi type then we only need bypass and not stall. But if we have LD, JAL, JALR as our current instruction, then we need to stall the next instruction for 2 clock cycles. After 2 clock cycle, these instructions will be in the WB stage and hence we can then use the result of current instruction as an operand for the next waiting instruction. For the 2 cycle the stall signal will be asserted which will hold the instructions.

2.6.2 Branch Prediction Unit

When a program runs certain flow of instruction is fetched and executed in pipeline. However, when a branch instruction is fetched, it may change the flow of the instructions. The execution result of branch instructions determines whether to change the flow in the execution stage, which is in the middle of the pipeline stage. If it changes its flow, the next instructions that are already being executed by other hardware resources prior to the execution stage become useless, which leads to a pipeline flush and degrades the processing speed. Since the branch instructions take more than 20 % of all instructions when running a program, this pipeline flush becomes a huge performance degradation factor in processors.

Branch prediction unit contains two different blocks: *branch direction predictor and branch target buffer (BTB)*. When instruction is fetched branch prediction unit will identify whether it is branch instruction. If the instruction is branch, the direction predictor will give the prediction result (Taken / Not Taken). If the branch is taken then the PC is updated by the target address obtained from BTB. The branch prediction unit is usually placed in instruction fetch stage. So, when prediction is correct the next instruction fetched after the branch instruction is from the correct instruction flow and pipeline can execute these instruction without any pipeline flush. But when prediction is wrong, the instruction executed after the branch needs to be flushed.

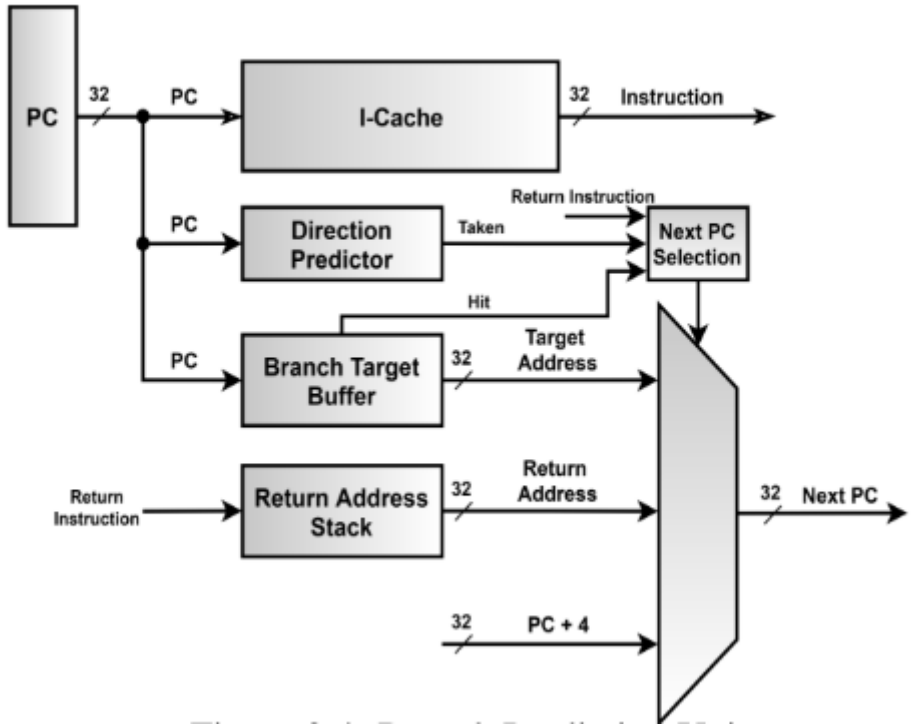

Figure 2.3: Branch Prediction Unit

2.7 Chisel Hardware Description Language

The dominant traditional hardware-description languages (HDLs), Verilog and VHDL, were originally developed as hardware simulation languages, and were only later adopted as a basis for hardware synthesis. Because the semantics of these languages are based around simulation, synthesizable designs must be inferred from a subset of the language, complicating tool development and designer education. Recent extensions such as SystemVerilog improve the type system and parameterized generate facilities but still lack many powerful programming language features.

An alternative approach is to begin from a domain-specific application programming language from which a hardware block is generated. Esterel[13] uses event-based statements to program hardware for reactive systems. DIL [15] is an intermediate language targeted at stream processing and hardware virtualization. Bluespec[14] supports a general concurrent computation model, based on guarded atomic actions.

Chisel[16] *(Constructing Hardware In a Scala Embedded Language)*, a new hardware design

language we have developed based on the Scala programming language[17]. Chisel is intended to be a simple platform that provides modern programming language features for accurately specifying low-level hardware blocks, but which can be readily extended to capture many useful high-level hardware design patterns. Chisel can generate fast cycle-accurate C++ simulators for a design, or generate low-level Verilog suitable for either FPGA emulation or ASIC synthesis with standard tools.

A key motivation for embedding Chisel in Scala is to support highly parameterized circuit generators, a weakness of traditional HDLs. We'll see some of the example chisel codes to get an idea how the use of chisel over standard HDLs like verilog and system Verilog can help us in quick development of our design.

2.7.1 Cache Generator

In Chisel, the basic configuration options can first be defined:

```
object CacheParams {
        val DIR-MAPPED =0
        val SET_ASSOC  =1
        val WRITE_THRU =0
        val WRITE_BACL =1
}
```

The main body of the cache generator component can then be declared with desired generator parameters and optional default values. The io bundle then references two IO interface bundles, one specifying a connection to a CPU and the other defining the memory interface. Computed parameters are then defined, followed by the main body of the generator:

```
class Cache(cache_type: Int = DIR_MAPPED,
        associativity: Int = 1,
        line_size: Int = 128,
        cache_depth: Int = 16,
        write_policy: Int = WRITE_THRU
        ) extends Component {
        val io = new Bundle() {
                val cpu = new IoCacheToCPU()
                val mem = new IoCacheToMem().flip()
                }
```

```
        val addr_idx_width = ( log(cache_depth) / log(2) ).toInt
        val addr_off_width = ( log(line_size/32) / log(2) ).toInt
        val addr_tag_width = 32 - addr_idx_width - addr_off_width - 2
        val log2_assoc = ( log(associativity) / log(2) ).toInt
        ...
        if (cache_type == DIR_MAPPED)
                ...
        }
```

The resulting Cache generator can then be used in a larger system:

```
val data_cache = new Cache(cache_type = SET_ASSOC, line_size = 64)
connection_to_cpu <> data_cache.io.cpu
connection_to_mem <> data_cache.io.mem
```

This cache can be connected to CPU and Main memory using bus interfaces.

2.7.2 Memory

Chisel defines a memory abstraction that can map to either simple Verilog behavioral descriptions, or to instances of memory modules that are available from external memory generators provided by foundry or IP vendors.

Chisel allows memory to be defined with a single write port and multiple read ports as follows:

```
Mem(depth: Int,
        target: Symbol = default, readLatency: Int = 0)
```

where **depth** is the number of memory locations, **target** is the type of memory used, readLatency is the latency of read ports to be defined on the memory. A memory object can then be read from using the **read(rdAddress)** method. For example, an audio recorder could be defined as follows:

```
def audioRecorder(n: Int) = {
        val addr = counter(UFix(n));
        val ram = Mem(n).write(button(), addr)
        ram.read(Mux(button(), UFix(0), addr))
}
```

where a counter is used as an address generator into a memory. The device records while button is **true**, or plays back when **false**.

We can use simple memory to create register files. For example we can make a one write port, two read port register file with 32 registers as follows:

```
val regs = Mem(32)
regs.write(wr_en, wr_addr, wr_data)
val idat = regs.read(iaddr)
val mdat = regs.read(maddr)
```

where a new read port is created for each call to read.

We can also use ***SyncReadMem*** to create synchronus memories.

```
class ReadWriteSmem extends Module {
        val width: Int = 32
        val io = IO(new Bundle {
                val enable = Input(Bool())
                val write = Input(Bool())
                val addr = Input(UInt(10.W))
                val dataIn = Input(UInt(width.W))
                val dataOut = Output(UInt(width.W))
        })
        val mem = SyncReadMem(1024, UInt(width.W))
        mem.write(io.addr, io.dataIn)
        io.dataOut := mem.read(io.addr, io.enable)
}
```

This corresponding piece of code in chiel generates a verilog code as given below:

```
  always @(posedge clock) begin
        if (mem_MPORT_en & mem_MPORT_mask) begin
                mem[mem_MPORT_addr] <= mem_MPORT_data;
        end
        mem_io_dataOut_MPORT_en_pipe_0 <= io_enable;
        if (io_enable) begin
                mem_io_dataOut_MPORT_addr_pipe_0 <= io_addr;
        end
  end
```

The timing diagram for the following memory is shown below:

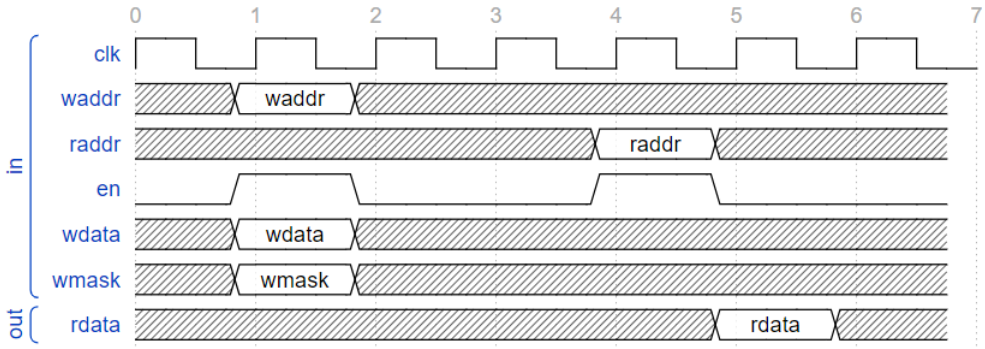

Figure 2.4: Synchronous Memory timing

2.8 Conclusion

In the above study report we have compared various industrial and academic RISC-V designs. RISC-V ISA has been studied in order to understand the software-hardware interaction and hardware can be better designed so as to deal with some anomalies at hardware level only (like hazards, stall) and make the compiler independent of the hardware, this can speed up the software development process. Then we proposed our RISCV architecture with various design blocks and strategies that we developed after going through various literature. For handling the hazards we'll be going with data bypass/forwarding and 2-bit branch prediction scheme which is commonly used in lot of other RISC-V designs across academia. We'll be designing this processor in the chisel Hardware descriptive language, and we have given some instances on how to design memory, cache efficiently with chisel which otherwise would have taken a lot more time and effort for doing the same job in Verilog HDL. Hence we have covered the basics of designing the RISC-V processor and we can start the design phase with lot more understanding.

Bibliography

[1] RISC-V International: https://riscv.org/

[2] BBC Research: https://www.bccresearch.com/market-research/semiconductor-manufacturing/global-risc-v-technology-market.html

[3] https://semico.com/content/risc-v-cpu-market-analysis-sip-socs-ai-and-design-starts

[4] https://riscv.org/technical/specifications/

[5] https://wiki.riscv.org/display/HOME/RISC-V+Technical+Specifications

[6] A. Waterman, Y. Lee, D. A. Patterson, K. Asanovic, V. I. U. level Isa, A. Waterman, Y. Lee, and D. Patterson, “The risc-v instruction set manual,” 2014.

[7] K. Asanovi and D. A. Patterson, “Instruction sets should be free: The case for risc-v,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146, Aug 2014. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2014/EECS-2014-146.html

[8] A. Waterman, K. Asanovic and SiFive Inc., “The RISC-V Instruction Set Manual Volume I: Unprivileged ISA,” Document Version 20191213, EECS Dept, University of California, Berkeley, CA, USA, December 13, 2019

[9] B. Zimmer et al., "A RISC-V Vector Processor With Simultaneous-Switching Switched-Capacitor DC–DC Converters in 28 nm FDSOI," in IEEE Journal of Solid-State Circuits, vol. 51, no. 4, pp. 930-942, April 2016, doi: 10.1109/JSSC.2016.2519386.

[10] Milanesi, L., 2023. Study and Development of RISC-V Mitigation Approach Based on Redundancy (Doctoral dissertation, Politecnico di Torino).

[11] I. Thanga Dharsni, K. S. Pande and M. K. Panda, "Optimized Hazard Free Pipelined Architecture Block for RV32I RISC-V Processor," 2022 3rd International Conference on

Smart Electronics and Communication (ICOSEC), Trichy, India, 2022, pp. 739-746, doi: 10.1109/ICOSEC54921.2022.9952122.

[12] https://passlab.github.io/CSE564/notes/lecture09_RISCV_Impl_pipeline.pdf

[13] Berry, G., and Gonthier, G. The Esterel synchronous programming language: Design, semantics, implementation. Science of Computer Programming 10, 2 (1992).

[14] Bluespec Inc. Bluespec(tm) SystemVerilog Reference Guide: Description of the Bluespec SystemVerilog Language and Libraries. Waltham, MA, 2004

[15] Goldstein, S., and Budiu, M. Fast compilation for pipelined reconfigurable fabrics. ACM/FPGA Symposium on Field Programmable Gate Arrays (1999).

[16] Jonathan Bachrach, Huy Vo, Brian Richards, Yunsup Lee, Andrew Waterman, Rimas Avižienis, John Wawrzynek, and Krste Asanović. 2012. Chisel: constructing hardware in a Scala embedded language. In Proceedings of the 49th Annual Design Automation Conference (DAC '12). Association for Computing Machinery, New York, NY, USA, 1216–1225. https://doi.org/10.1145/2228360.2228584

[17] Odersky, M. e. a. Scala programming language. http://www.scala-lang.org/.